Aerial Object Classification Project Report

---

1. Introduction

The objective of this project is to develop a deep learning-based system capable of classifying aerial images into two categories: Bird and Drone. This problem is highly relevant in domains such as airport safety, surveillance systems, wildlife monitoring, and defence applications, where accurate differentiation between birds and drones is crucial.

With the increasing use of drones and the risks associated with bird strikes in aviation, an automated classification system can significantly improve safety and decision-making.

---

2. Problem Statement

The goal is to build a binary image classification model that can:

- **Accurately classify aerial images as either Bird or Drone**
- **Reduce false positives in surveillance systems**
- **Improve detection reliability in real-world scenarios**

---

3. Dataset Description

The dataset used in this project consists of labelled aerial images categorized into two classes:

- **Bird**
- **Drone**

The dataset is structured into three subsets:

- **Training Set**
- **Validation Set**
- **Test Set**

Dataset Distribution:

Dataset	Bird Images	Drone Images
Train	1414	1248
Valid	217	225
Test	121	94

This balanced distribution ensures fair training and evaluation.

---

4. Exploratory Data Analysis (EDA)

4.1 Data Structure

The dataset follows a directory-based structure:

```
train/
  ├ bird/
  ├ drone/
valid/
  ├ bird/
  ├ drone/
test/
  ├ bird/
  ├ drone/
```

This structure is compatible with Keras' flow from directory method.

4.2 Class Distribution Analysis

- The dataset is relatively balanced between bird and drone images.
- Slight imbalance exists but does not significantly affect training.

4.3 Image Characteristics

- Images are RGB format (.jpg)
- Varying resolutions and lighting conditions
- Diverse backgrounds (sky, trees, urban areas)

4.4 Data Visualization

Sample images were visualized to understand variations in shape, texture, and orientation. This helped in designing augmentation strategies.

---

5. Data Preprocessing

Data preprocessing is a crucial step to prepare the dataset for model training.

Steps Applied:

- Rescaling: Pixel values normalized from [0, 255] to [0, 1]

- **Resizing: All images resized to 224 × 224**
- **Batch Processing: Data loaded in batches using generators**

---

6. Data Augmentation

To improve model generalization and prevent overfitting, the following augmentation techniques were applied:

- **Rotation**
- **Zoom**
- **Horizontal flipping**

This increases dataset diversity without collecting new data.

---

7. Model Development

Two models were developed and compared:

7.1 Custom CNN Model

A Convolutional Neural Network was built from scratch using:

- **Convolutional layers (feature extraction)**
- **MaxPooling layers (dimensionality reduction)**
- **Fully connected layers (classification)**

Advantages:

- **Full control over architecture**

Limitations:

- **Requires more data and training time**

7.2 Transfer Learning (MobileNetV2)

A pretrained model MobileNetV2 was used with transfer learning.

Key Features:

- **Pretrained on ImageNet dataset**
- **Lightweight and efficient**
- **Uses depthwise separable convolutions**

Approach:

- **Base model layers frozen**
- **Custom classification layers added**

Advantages:

- **Faster training**
- **Higher accuracy**
- **Better feature extraction**

---

8. Model Training

Both models were trained using:

- **Training dataset**
- **Validation dataset for monitoring**

Key Parameter:

- **Epochs: 10**

Definition:

An epoch represents one complete pass through the entire training dataset.

Training Process:

- **Model learns patterns iteratively**
- **Weights updated using backpropagation**
- **Validation used to monitor performance**

---

9. Model Evaluation

Model performance was evaluated using multiple metrics:

9.1 Accuracy

Measures overall correctness of predictions.

9.2 Precision

Measures how many predicted positives are correct.

9.3 Recall

Measures how many actual positives are correctly identified.

9.4 F1-Score

Harmonic mean of precision and recall.

9.5 Confusion Matrix

Used to visualize:

- **True Positives**
- **False Positives**
- **True Negatives**
- **False Negatives**

9.6 Performance Graphs

- **Accuracy vs Epochs**
- **Loss vs Epochs**

These graphs help identify overfitting and training behavior.

---

10. Model Comparison

Model	Accuracy	Performance
Custom CNN	Lower	Basic feature learning
MobileNetV2	Higher	Better generalization

Conclusion:

MobileNetV2 outperformed the custom CNN due to:

- **Pretrained knowledge**
- **Better feature extraction**
- **Faster convergence**

---

11. Deployment (Streamlit)

A web application was developed using Streamlit to provide an interactive interface.

Features:

- **Image upload functionality**
- **Real-time prediction**
- **Confidence score display**

Workflow:

1. **Upload image**
2. **Preprocess image**
3. **Predict using trained model**
4. **Display result**

---

12. Real-World Applications

- **Airport bird-strike prevention**
- **Drone detection in restricted areas**
- **Wildlife monitoring**
- **Surveillance systems**

---

13. Limitations

- **Works only on image classification (no object detection)**
- **Performance depends on dataset quality**
- **Cannot detect multiple objects in one image**

---

14. Future Improvements

- **Implement YOLO for object detection**
- **Use real-time video input**
- **Improve dataset diversity**
- **Deploy on cloud platform**

---

15. Optional YOLOv8 Object Detection Approach

In addition to image classification, this project can be extended using YOLOv8 for real-time object detection. Unlike classification, object detection identifies both the object class and its location using bounding boxes.

15.1 Proposed Workflow:

1. **Install YOLOv8 using Ultralytics package**
2. **Use the provided object detection dataset with annotations**
3. **Configure data.yaml with dataset paths and classes**
4. **Train the model using pretrained YOLO weights**
5. **Validate model performance**
6. **Run inference on new images/videos**
7. **Display Bird or Drone with confidence score and bounding boxes**

15.2 Benefits:

- **Real-time surveillance capability**
- **Multiple object detection in one frame**
- **Better for CCTV/drone camera integration**

15.3 Future Scope:

This can be integrated into airport monitoring, defence surveillance, and smart security systems.

---

16. Conclusion

This project successfully demonstrates an end-to-end deep learning pipeline for aerial image classification.

Key achievements:

- **Built and compared CNN and MobileNetV2 models**
- **Achieved high classification performance**
- **Deployed model using Streamlit**

The system can be extended for real-time applications in surveillance and safety systems.

---

17. Technologies Used

- **Python**
- **TensorFlow / Keras**
- **NumPy**
- **Matplotlib / Seaborn**
- **Streamlit**

---

18. References

- **TensorFlow Documentation**
- **Keras Documentation**
- **ImageNet Dataset**
- **Streamlit Documentation**